Jak začít programovat v jazyce python

(minimální sada příkazů pro U3V)

Jak otevřít prostředí Google Colab

Nejjednoduší metoda jak psát malé simulace v pythonu je využít prostředí Google Colab.V Google Colab https://colab.google.com se program spoušti ve Web prohlížeči (Chrome Edge, Safari) a vše je připraveno pro naše účely bez instalcí a nastavování.

- Jediné, co je nutné je mít Google účet. Pokud máte Android telefon (nebo tablet) tak jej již máte, pokud jej někdo nemá může si jej založit zadarmo na https://www.google.com/intl/cs/account/about/ (tlačítko "Vytvoření účtu")
- Počítač musí být připojen k internetu - protože vše se odehrává v internetovém prohlížeči a vytvářené programy beží na severu Googlu (ale v případě potřeby lze vše stahnout na lokální PC a pokračovat tam).

Par zakladnich pravidel jak spoustet programy v Google COLAB

- program se zde zapisuje do bunek (nekolik radek kodu), bunka kodu se prida pomoci horniho menu (**+Code**), bunky lze posouvat (po kliknuti se objevi sipky nahoru a dolu, kterymi se bunka posouva)
- program v bunce se spusti kliknutim na trojuhelnik vlevo (na okraji bunky) nebo **Ctrl Enter** (stisknuti 2 klaves najednou), pokud bunka generuje nejaky vystup zobrazi se pod bunkou
- lze take vybrat jen cast bunky (pomoci mysi) a po stisku praveho tlacika mysi se objevi volba **Run Selection** - nebo lepe pomoci klavesove zkratky **Shift Ctrl Enter** (stisknuti 3 klaves najednou)
- vystup se generuje bud prikazem **print** (nebo v pripade grafu prikazy **plot**)
- pokud bunka na konci obsahuje jen promennou pak se vystup tak zobrazi hodnota
- jednotlive bunky lze spoustet samostatne v jakekoliv poradi, stav promenych se "pamatuje" mezi bunkami (pozor pokud napr. nejaka promenna neni definovana a chceme ji pouzit v jine bunce tak dojde k chybe)
- bezpecne overeni ze vse je spravne je spustit vsechny bunky pomoci horniho menu (**|>Run All**)
- pokud si nejsme jisti tak vedle (**|>Run All**) je sipka dolu, ktery rozbali menu obsahujici (**Restart Session**), jehoz vyber vymaze vsechny promenne a nastavi cely program do pocatecniho stavu
- menu otevirane sipkou dolu vedle (**|>Run All**) take obsahuje (**Clear All Outputs**) ktere smaze vystupy vsech bunek

Elementarni pravidla python

- jmena promennych a klicovych slov jazyka python rozlisuji velka a mala pismena (**var_A** a **VAR_A** jsou ruzne promenne, **FOR** neni spravne klicove slovo pro prikaz cyklu (python ma vsechny prikazy malymi pismeny)
- komentare v programu zacinaji # a pak vse do konce radky je komentar, **Ctrl /** zakomentuje cely radek (nebo vsechny oznacene radky)
- python kod obsahuje bloky (napr. v prikazech if else), ktere musi mit shodne odsazeni (napr pomoci tabulatoru), pokud format neni spravne bunky to indikuji cervenym podtrzenim
- dole je na stavovem radku **{} Variables** ktere otevre okno zobrazujici hodnoty promennych, lze tak videt jak se meni
- operatory
  - mocnina ** napr 2**4
  - porovnani dvou cisel (nebo textu)
    - vetsi,mensi,vetsi nebo rovno, mensi nebo rovno >,<,>=,=<
    - rovna se ==
    - nerovna se !=
  - logicke operatory
    - x = a or b and c # nejdříve se vyhodnotí and, až poté or
    - x = a or (b and c) # to stejné jako předchozí, jen čitelněji zapsáno
    - x = (a or b) and c # tentokrát se nejdřív vyhodnotí or
    - x = not a or b # nejdříve not, až potom or
    - x = not (a or b) # nejdříve or, potom not

Elementární příkazy pythonu

```
# Nastaveni hodnot ciselnych promennych
K=3  # vznikne promenna K  a jeji hodnota se nastavi na 3 (cele cislo)
print(K)  # po kliknuti na trojuhelnik vlevo se kod spusti vypise hodnotu A dole
P=0.5     # proměnná P je racionalni cislo
print(P)
```

```
3
0.5
```

```
K=5 # prenastav K
K   # Nova hodnota je na konci bunky tak se jeji hodnota zorazi
```

```
5
```

```
P1=0.1 # nastavim novou promennou 1
# a nyni jen vypisu hodnotu P1 tim, ze ji dam na konec bunky
P1
```

```
0.1
```

```
# Nastaveni textove promenne
Jmeno="Josef Novák"  # promenna Jmeno obsahuje text
print(Jmeno)
```

```
Josef Novák
```

```
# Elementarni vypocty
K=K+1         # prida 1 k hodnote promenne K
print(K)
Px=P/10+K/100 # vypocte vyraz do promenne Px
print(Px)
```

```
6
0.11
```

```
# tisk vice veci najednou (2 promenne a vyraz)
print("K=",K,"P=",P, 1+3*K)
```

```
K= 6 P= 0.5 19
```

```
# vypocet mocniny
2**10        # 2 na 10-tou
```

```
1024
```

```
M=17
J=5
L=M % J   # 17=3*5+2
print("zbytek:",L) # vypocet zbytku po deleni celych cisel (operator %)
K=M // J
print("celociselne deleni:",K)
```

```
zbytek: 2
celociselne deleni: 3
```

```
# elementarni funkce
#  otestuj pomoci vyberu radek oddelenych prazdnym radkem a stiskem Shift Ctrl Enter
x=-3
y=2.5
z=11.3
abs(x)        # absolutní hodnota x

min(x, y)     # minimum z x, y

max(x, y, z)  # maximum z x, y
```

```
11.3
```

```
# zakladni konstaty
import math  # tohle je nutno dat nekam na zacatek programu, aby python zacal znat matematicke funkce
```

```
print(math.pi)  # 3.141592653589793   hodnota konstanty pi z math
print(math.e)   # 2.718281828459045   Eulerova konstanta z math
```

```
3.141592653589793
2.718281828459045
```

```
# nekolik zakladnich matematickych funkci
import math
print("Odmocnina 2:",math.sqrt(2))       # odmocnina
print("sin 45:",math.sin(45/180*math.pi)) # sin (45 deg)  Operand musi byt v radianech
print("cos 60:",math.cos(60/180*math.pi)) # cos (60 deg)

# lepe
# preved stupne na radiany pomoci funkce  radians
result_sin = math.sin(math.radians(45))
print(result_sin)
result_cos = math.sin(math.radians(60))
print(result_cos)
```

```
Odmocnina 2: 1.4142135623730951
sin 45: 0.7071067811865475
cos 60: 0.5000000000000001
0.7071067811865475
0.8660254037844386
```

```
# generator nahodnych cisel
from random import *  # tohle je nutne  aby python zacal znat nahodne generatory
u=random()  # nahodne cislo z intervalu [0,1)
print(u)
```

```
0.4736963196173737
```

```
# pole (vektor) - sada promenych adresovanych indexem
# tohle je nutno dat nekam na zacatek programu, aby python zacal znat pole a vektory
import numpy
A=numpy.array([1,2,3,4,5])  # pole definovane seznamem
print(A)

# definuj pole delky NMC=5 a nastav jeho hodnoty na 0
NMC=5
B=numpy.zeros(NMC)
print(B)
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
```

```
# elementy pole (!! pozor cislovani elementu zacina od 0)
print(A[0])
print(A[4])
A[0]=A[0]+A[4]*3
print(A[0])
```

```
1
5
16
```

```
# podmineny prikaz (porovnani >,<,>=,=<,==,!=)
K=8
if K >= 2:
  print("K je vetsi nebo rovno nez 2")
else:
  print("K je mensi nez 2")
```

```
K je vetsi nebo rovno nez 2
```

```
# Prikaz if vyhodnocuje logickou podminku - hodnotu true nebo false (pravda nepravda)
#  otestuj pomoci vyberu radek oddelenych prazdnym radkem a stiskem Shift Ctrl Enter

a = True    # pravda
b = False   # nepravda
c = a or b  # nebo -> True
c

d = a and b # a zároveň -> False
d

c = not a   # negace -> False
c
```

```
False
```

```
# porovnani vraci logickou hodnotu (https://www.umimeinformatiku.cz/programovani-logicke-vyrazy)
#  otestuj pomoci vyberu radek oddelenych prazdnym radkem a stiskem Shift Ctrl Enter

x = 42     # přiřazení do proměnné
x == 42    # test rovnosti

x != 42    # test nerovnosti

x > 42     # je větší

x >= 42    # je větší nebo rovno
```

```
True
```

```
# cyklus - tiskni cisla od 0 do 5 a jejich druhe mocniny
NMC=5
for i in range(1,NMC+1):      # range = rozsah od 1 az dokud i neni rovno NMC+1
  i2=i*i
  r01=random()
  print("i=",i,"i2=",i2,"random cislo 0..1:",r01)
```

```
i= 1 i2= 1 random cislo 0..1: 0.2446677501110588
i= 2 i2= 4 random cislo 0..1: 0.9906263280815946
i= 3 i2= 9 random cislo 0..1: 0.8681550246881401
i= 4 i2= 16 random cislo 0..1: 0.8526514477446667
i= 5 i2= 25 random cislo 0..1: 0.26355402181800747
```

```
# podmineny prikaz v cyklu
for i in range(1,NMC+1):      # range = rozsah od 1 az dokud i neni rovno NMC+1
    r01=random()
    print("i=",i,"random cislo 0..1:",r01)
    if (i % 2) == 0:        # test zdali i je sude cislo ( zbytek po deleni i dvemi je 0)
        print("i je sude")
    else:
        print("K je liche")
print("konec cyklu")
```

```
i= 1 random cislo 0..1: 0.8853940278723723
K je liche
i= 2 random cislo 0..1: 0.12109809005883765
i je sude
i= 3 random cislo 0..1: 0.4565341899302042
K je liche
i= 4 random cislo 0..1: 0.035390872970394116
i je sude
i= 5 random cislo 0..1: 0.5589080451474993
K je liche
konec cyklu
```

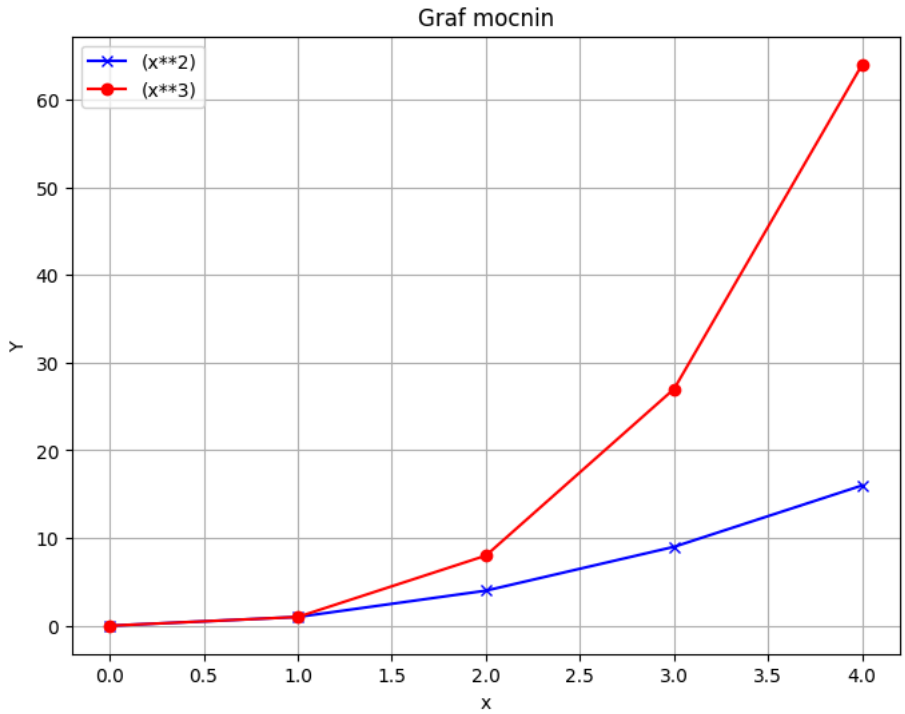
```
#malovani grafu

import numpy  # tohle je nutno dat nekam na zacatek programu, aby python zacal znat pole a vektory
import matplotlib.pyplot as plt # tohle je nutno dat nekam na zacatek programu, aby python zacal znat malovani grafu

# nadefinuj pole souradnic
x=numpy.zeros(5)
y2=numpy.zeros(5)
y3=numpy.zeros(5)

# x=numpy.array([1,2,3,4,5,])  # lze napsat rucne jako pole definovane seznamem
# y=numpy.array([1,4,9,16,25])  # lze napsat rucne jako pole definovane seznamem
for i in range(5):
  x[i]=i
  y2[i]=i*i
  y3[i]=i**3

# nadefinuj graf
plt.figure(figsize=(8, 6))      #velikost grafu v palcich
plt.plot(x, y2, label='(x**2)', color='blue',marker='x') # prvni cara
plt.plot(x, y3, label='(x**3)', color='red',marker='o') # druha cara
plt.title('Graf mocnin')
plt.xlabel('x')
plt.ylabel('Y')
plt.grid(True)      # pridej mrizku
plt.legend()      # pridej popisku car
plt.show()      # zobraz nadefinovane
```


Graf mocnin
(x**2)
(x**3)
Y
x
0
10
20
30
40
50
60
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0